Darshan Abinav RK 241501039

13. Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Aim

To analyse the technical, architectural and operational limitations of low-code platforms when they are used for complex, large-scale enterprise requirements.

Theory

- **Customisation limits:** visual components and templates restrict special algorithms, unusual user interfaces and deep back-end logic.
- **Scalability and performance:** shared platform resources, call limits and automatic database handling can become bottlenecks under very high volumes.
- **Vendor lock-in:** applications are built from platform-specific components and cannot easily be moved to another stack.
- **Integration with legacy systems:** mainframes and on-premise systems may need custom gateways that cancel the speed advantage.
- **Cost at scale:** subscription and API-call pricing can exceed the cost of a tuned custom system at high volume.

Case Study Scenario: Nationwide Retail Pricing and Inventory Engine

Requirement. A retail chain with thousands of stores wants one system that updates stock for every sale within a second, prices items with a demand-forecasting model and syncs with an old on-premise ERP.

Platform chosen. A hypothetical enterprise low-code platform, selected for quick prototyping and built-in database handling.

Limitations met during implementation

- **Customisation wall:** the pricing model could not be expressed with the platform's visual nodes and short scripts, so large custom script blocks were needed and became hard to maintain.
- **Data layer rigidity:** high-frequency writes timed out because indexes and query plans could not be tuned.
- **Integration:** the legacy ERP needed a custom API gateway, which removed most of the time saved.
- **Cost:** tier-based pricing and API-call limits made the platform costlier than a containerised microservice design as volume grew.

Limitations observed in my own application

- The notification emails in DS-04 and DS-06 go to one address written in the script; a field reference would be needed to mail the claimant.
- The duplicate check (DS-02) could match the record being edited; adding the condition ID != input.ID to the criteria is the planned fix (BUG-01).
- Matching lost items with found items automatically needs similarity logic that simple criteria cannot express, so it would need a pro-code extension.

- The application runs in a trial edition and depends on Zoho Creator features.
- Two approved claims are not reflected on their found items (BUG-02) and the dashboard totals disagree with the reports (BUG-03).

Procedure

1. Study the common limitations of low-code platforms.
2. Analyse a complex enterprise scenario against those limitations.
3. Relate the findings to the limitations seen in the lost and found application.
4. Conclude where a pro-code approach is needed.

Result

Low-code platforms deliver departmental and workflow applications quickly, but customisation limits, scalability ceilings, vendor lock-in, integration effort and cost at scale make traditional or hybrid development necessary for complex enterprise systems.